

18 | 自托管构建：如何使用 Tekton 构建镜像？

王炜 · 云原生架构与GitOps实战



你好，我是王炜。

前两节课，我们分别介绍了如何使用 GitHub Action Workflow 和 GitLab CI 来自动构建镜像，它们配置起来相对简单，作为平台自带的功能，也不需要花费额外的维护成本。并且，它们都和 Git 仓库深度整合，这就让触发流水线变得非常简单了。

对于构建次数较少的团队来说，在免费额度范围内使用它们是一个非常好的选择。但是对于构建次数非常频繁的中大型团队来说，综合考虑费用、可控和定制化等各方面因素，他们可能会考虑使用其他自托管的方案。

这节课，我们就来介绍其中一种自动构建镜像的自托管方案：**使用 Tekton 来自动构建镜像**。Tekton 是一款基于 Kubernetes 的 CI/CD 开源产品，如果你已经有一个 Kubernetes 集群，那么利用 Tekton 直接在 Kubernetes 上构建镜像是一个不错的选择。

我会首先带你了解 Tekton 的基本概念，然后我们仍然以示例应用为例，从零开始为示例应用配置构建镜像的流水线，并结合 GitHub 为 Tekton 配置 Webhook 触发器，实现提交代码之后触发 Tekton 流水线并构建镜像，最后推送到镜像仓库的过程。

在学完这节课之后，你将基本掌握 Tekton 的流水线以及触发器的用法，并具备独立配置它们的能力。

在开始今天的学习之前，你需要具备以下前提条件。

在本地安装了 kubectl。

已经按照 [第 16 讲](#) 将 [kubernetes-example](#) 示例应用代码推送到了自己的 GitHub 仓库中。

准备 Kubernetes 集群

由于我们在实践的过程中需要 Kubernetes 集群的 Loadbalancer 能力，所以，首先你需要准备一个云厂商的 Kubernetes 集群，你可以使用 AWS、阿里云或腾讯云等任何云厂商。这里我以开通腾讯云 TKE 集群为例演示一下，**这部分内容比较基础，如果你已经有云厂商 Kubernetes 集群，或者熟悉开通过程，都可以跳过这个步骤。**

首先，登录腾讯云并在 [这个页面](#) 打开 TKE 控制台，点击“新建”按钮，选择“标准集群”。

Serverless 集群

支持添加使用超级节点，无集群管理费用。快速配置大规格超级节点，即可部署海量安全沙箱容器，弹性使用近乎无限的容器资源，且仅需为实际运行的 Pod 资源付费，轻松承载高稳在线业务和批量计算业务，保障业务稳定运行的同时可明显降低资源费用。

• 适用于高稳定性、高安全性常驻业务及临时计算任务

• 支持安全沙箱容器，业务容器强隔离，无互相干扰

• 可秒级启动上万 Pod，并按实际时长计费

安全沙箱容器

支持超级节点

极速启动容器

标准集群

默认集群类型，完全兼容开源 Kubernetes 集群标准功能，并强化节点管理、集群网络、容器调度等能力。单个集群内可同时添加管理超级节点、原生节点、普通 CVM 节点及 IDC 内节点，可根据业务场景搭配使用，实现计算资源最大化利用。

• 全场景适用，完全兼容开源 Kubernetes 集群标准能力

• 同时支持 超级节点、原生节点、注册节点及 CVM

• 支持资源可视化及优化分析，轻松提升资源利用率

标准 K8s 集群

支持超级节点

支持原生节点

边缘集群

支持添加使用边缘节点，把数据中心 Kubernetes 集群能力快速延伸到边缘地域，并使用云原生方式进行资源管理和应用生命周期管理；同时具备独创的多地域边缘自治、流量闭环以及分布式健康状态检查能力

• 边缘计算，可通过云原生方式管理位于边缘的计算资源，如边缘服务器、边缘物联网设备等，解决弱网以及节点自治问题

• 多地域管理，可以使用单一集群统一纳管多地域资源，并实现地域内流量闭环

边缘计算

边缘云原生

多地域管理

流量闭环

注册集群

可以将用户本地基础设施的 Kubernetes 集群或者其他云厂商的 Kubernetes 集群注册到容器服务进行统一管理，并使用 云原生分布式云中心 产品实现多云多集群的统一管理。

• 多云管理，灵活接入和管理企业的各种计算资源

• 高可用容灾，统一治理多集群应用、流量、存储等

• 自动发布，可对接现有 DevOps 系统，实现多云发布

多云纳管

生态开源

高可用容灾

创建

取消

在创建集群页面输入“集群名称”，“所在地域”项中选择“中国香港”，集群网络选择“Default”，其他信息保持默认，点击下一步。

运行时组件

docker

containerd

如何选择

containerd 是更为稳定的运行时组件，支持 OCI 标准，不支持 docker api

所在地域

广州

上海

中国香港

多伦多

北京

新加坡

硅谷

成都

法兰克福

首尔

南京

雅加达

圣保罗

处在不同地域的云产品内网不通，购买后不能更换。建议选择靠近您客户的地域，以降低访问延时、提高下载速度。

集群网络

Default-VPC

↕

↻ CIDR: 172.19.0.0/16

如现有的网络不合适，您可以去控制台[新建私有网络](#)

容器网络插件

Global Router

VPC-CNI

如何选择

Global Router 是腾讯云 TKE 基于 VPC 路由实现的容器网络插件，可设置独立平行于 VPC 的容器网段。

容器网络

CIDR

172

16

0

0

16

使用指引

创建后不能修改

接下来进入到 Worker 节点配置阶段。在“机型”一栏中选择一个 2 核 8G 的节点，在“公网带宽”一栏中将带宽调整为 100Mbps，并且按量计费。

计费模式①

按量计费

包年包月

详细对比 [🔗](#)

Worker 节点配置

可用区①

香港二区

香港三区

节点网络①

Default-VPC

Default-Subnet

共4093个子网IP，剩4093个可用

CIDR:172.19.0.0/16

如现有的网络不合适，您可以去控制台[新建私有网络](#) [或新建子网](#) [🔗](#)

机型

SA2.MEDIUM8(标准型SA2.2核8GB) [🔗](#)

系统盘

通用型SSD云硬盘 50GB [🔗](#)

数据盘

暂不购买 [🔗](#)

公网带宽

按使用流量计费 100Mbps [🔗](#)

主机名

自动生成 [🔗](#)

云服务器数量

—

1

+

CVM配额:您当前云服务器个数配额为0/500，您最多可购买500台，您可以通过[提交工单](#) [🔗](#) 申请提升配额。

[▶ 高级设置](#)

确定

取消

点击“下一步”。后续的页面都保持默认选择，点击“完成”创建 Kubernetes 集群。这里需要等待几分钟直至集群准备就绪。

名称/ID	监控	状态 ▼	集群类型 ▼	kubernetes版本	节点数
docker-build cls-hx2jjous 🔗		创建中  管理规模L5 查看进度	托管集群	1.22.5	0台

共 1 条

点击集群名称“docker-build”进入集群详情页，在“集群 API Server 信息”一栏找到“外网访问”，点击开关来开启集群的外网访问。

集群API Server信息

外网访问 ☐ 未开启

内网访问 ☐ 未开启

[Kubeconfig权限管理](#)

在弹出的新窗口中，选择“Default”安全组并选择“按使用流量”计费，访问方式选择“公网IP”，然后点击“保存”开通集群外网访问。

i 开启外网访问, 会将集群apiserver暴露公网, 请谨慎操作。您需要通过安全组来配置来源授权, 我们会将您指定的安全组绑定到外网访问入口的CLB上, 达到访问控制的效果。

安全组 **i**

sg-nlm07319 | default



集群访问代理的流量默认走443端口, 请确认您选择的安全组内针对来源IP已放通443端口, 以确保开启功能后集群可以正常访问。

运营商类型

BGP

网络计费模式

按带宽计费

按使用流量

共享带宽包

带宽上限



1Mbps

512Mbps

1024Mbps

2048Mbps

— 10 + Mbps

访问方式

公网域名

公网IP

保存

取消

等待“外网访问”开关转变为启用状态。

接下来, 在“Kubeconfig”一栏点击“复制”, 复制集群 Kubeconfig 信息。

集群APIServer信息

外网访问



已开启

已设置安全组: [sg-nlm07319](#)

访问ip

101.32.56.36:443

复制

KubeConfig

复制

下载

内网访问



未开启

接下来，将集群证书信息内容写入到本地的 `~/.kube/config` 文件内，这是 `kubectl` 默认读取 `kubeconfig` 的文件位置。

为了避免覆盖已有的 `kubeconfig`，首先你需要备份 `~/.kube/config` 文件。

```
1 $ mv ~/.kube/config ~/.kube/config-bak
```

 复制代码

然后新建 `~/.kube/config` 文件，将刚才复制的 Kubeconfig 内容写入到该文件内。

最后，执行 `kubectl get node` 来验证 `kubectl` 与集群的联通性。

```
1 $ kubectl get node
2 NAME          STATUS    ROLES    AGE    VERSION
3 172.19.0.107   Ready    <none>   5m21s   v1.22.5-tke.5
```

 复制代码

待 Node 信息返回后，我们的 Kubernetes 集群也就准备好了。

安装组件

准备好云厂商 Kubernetes 集群之后，接下来我们需要安装两个组件，分别是 Tekton 相关的组件以及 Ingress-Nginx。

Tekton

首先，安装 Tekton Operator。

 复制代码

```
1 $ kubectl apply -f https://storage.googleapis.com/tekton-releases/pipeline/latest
2 namespace/tekton-pipelines created
3 podsecuritypolicy.policy/tekton-pipelines created
4 .....
```

等待 Tekton 所有的 Pod 就绪。

 复制代码

```
1 $ kubectl wait --for=condition=Ready pods --all -n tekton-pipelines --timeout=300
2 pod/tekton-pipelines-controller-799f9f989b-hxmlx condition met
3 pod/tekton-pipelines-webhook-556f9f7476-sgx2n condition met
```

接下来，安装 Tekton Dashboard。

 复制代码

```
1 $ kubectl apply -f https://storage.googleapis.com/tekton-releases/dashboard/latest
2 serviceaccount/tekton-dashboard created
3 role.rbac.authorization.k8s.io/tekton-dashboard-info created
4 .....
```

然后，分别安装 Tekton Trigger 和 Tekton Interceptors 组件。

 复制代码

```
1 $ kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest
2 podsecuritypolicy.policy/tekton-triggers created
3 clusterrole.rbac.authorization.k8s.io/tekton-triggers-admin created
4 .....
5
6 $ kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest
7 deployment.apps/tekton-triggers-core-interceptors created
8 service/tekton-triggers-core-interceptors created
9 .....
```

等待所有 Tekton 的所有组件的 Pod 都处于就绪状态，Tekton 就部署完成了。

 复制代码

```
1 $ kubectl wait --for=condition=Ready pods --all -n tekton-pipelines --timeout=300s
2 pod/tekton-dashboard-5d94c7f687-8t6p2 condition met
3 pod/tekton-pipelines-controller-799f9f989b-hxmlx condition met
4 pod/tekton-pipelines-webhook-556f9f7476-sgx2n condition met
5 pod/tekton-triggers-controller-bffdd47cf-cw7sv condition met
6 pod/tekton-triggers-core-interceptors-5485b8bd66-n9n2m condition met
7 pod/tekton-triggers-webhook-79ddd8d6c9-f79tg condition met
```

Ingress-Nginx

安装完 Tekton 之后，我们再来安装 Ingress-Nginx。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/kubernetes/ingress-nginx/master/deploy/quickstart/quickstart.yaml
2 namespace/ingress-nginx created
3 serviceaccount/ingress-nginx created
4 serviceaccount/ingress-nginx-admission created
5 .....
```

等待所有 Ingress-Nginx Pod 处于就绪状态，Ingress-Nginx 就部署完成了。

 复制代码

```
1 $ kubectl wait --for=condition=AVAILABLE deployment/ingress-nginx-controller --all
2 deployment.apps/ingress-nginx-controller condition met
```

暴露 Tekton Dashboard

配置好 Tekton 和 Ingress-Nginx 之后，为了方便访问 Tekton Dashboard，我们要通过 Ingress 的方式暴露它，将以下内容保存为 tekton-dashboard.yaml。

 复制代码

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: ingress-resource
5   namespace: tekton-pipelines
6   annotations:
7     kubernetes.io/ingress.class: nginx
8     nginx.ingress.kubernetes.io/ssl-redirect: "false"
9 spec:
10  rules:
11    - host: tekton.k8s.local
12      http:
13        paths:
14          - path: /
15            pathType: Prefix
16            backend:
17              service:
18                name: tekton-dashboard
19                port:
20                  number: 9097
```

然后，执行 `kubectl apply` 将它应用到集群内。

 复制代码

```
1 $ kubectl apply -f tekton-dashboard.yaml
2 ingress.networking.k8s.io/ingress-resource created
```

接下来，获取 Ingress-Nginx Loadbalancer 的外网 IP 地址。你可以使用 `kubectl get service` 来获取。

 复制代码

```
1 $ kubectl get services --namespace ingress-nginx ingress-nginx-controller --output
2 43.135.82.249
```

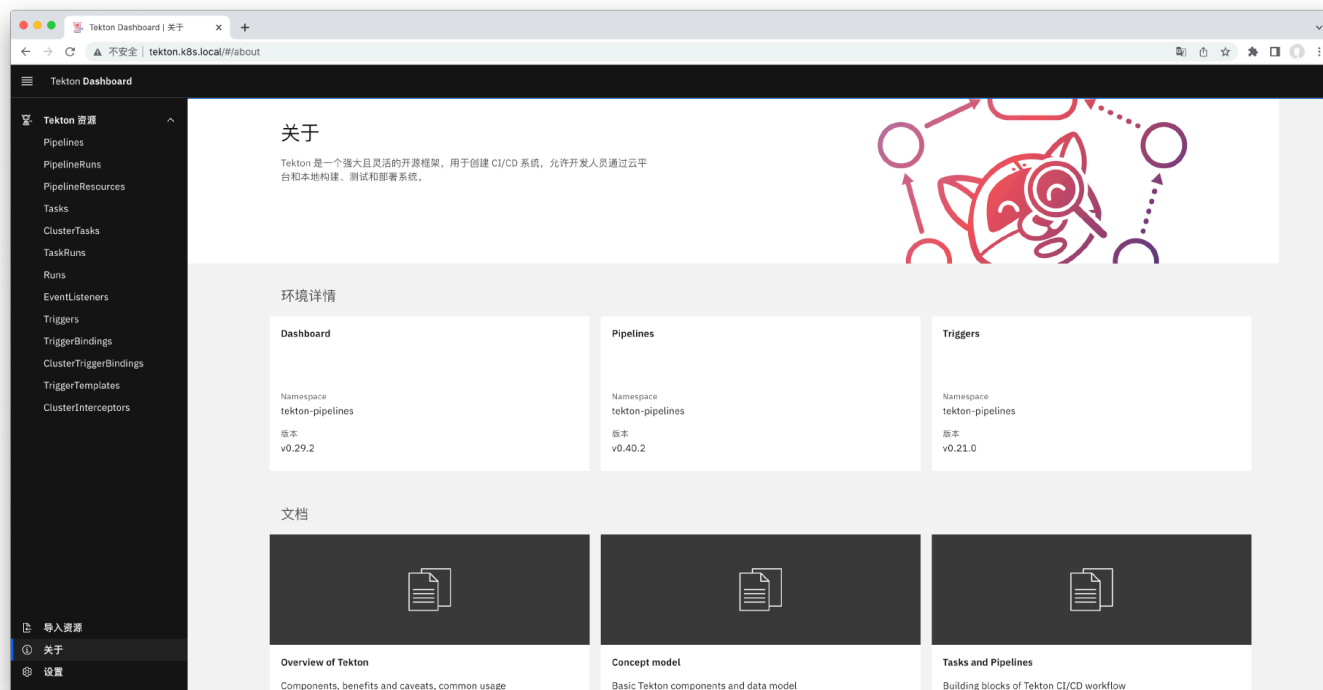
由于之前我在 Ingress 策略中配置的是一个虚拟域名，所以需要在本机配置 **Hosts**。当然你也可以将 Ingress 的 host 修改为实际的域名，并且为域名添加 DNS 解析，也能达到同样的效果。

以 Linux 系统为例，要修改 Hosts，你需要将下面的内容添加到 `/etc/hosts` 文件内。

 复制代码

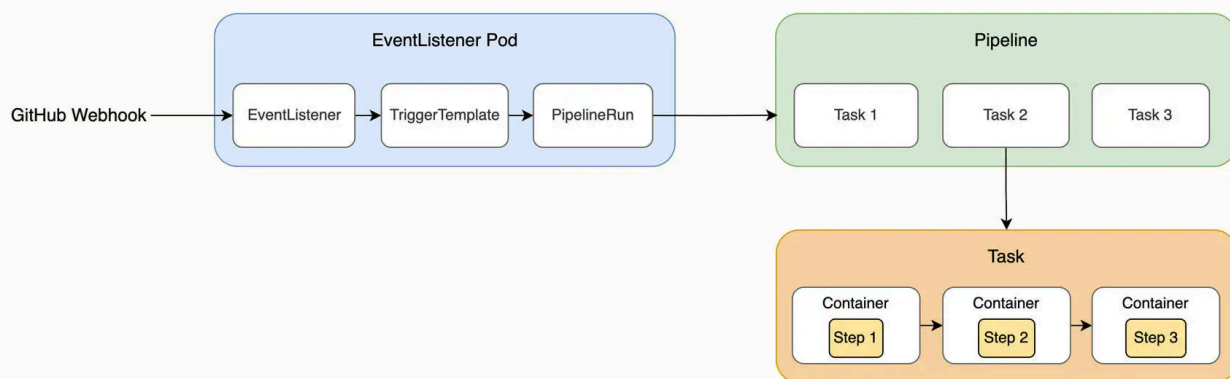
```
1 43.135.82.249 tekton.k8s.local
```

然后，你可以通过域名 <http://tekton.k8s.local> 来访问 Tekton Dashboard 了。



Tekton 简介

在正式创建 Tekton 流水线之前，你需要先了解一些基本概念，你可以结合下面这张图来理解一下。



在上面这张图中，从左往右依次出现了这几个概念：EventListener、TriggerTemplate、PipelineRun、Pipeline、Task 以及 Step，接下来，我就简单介绍一下它们。

EventListener

EventListener 顾名思义是一个事件监听器，它是外部事件的入口。EventListener 通常以 HTTP 的方式对外暴露，在我们这节课的例子中，我们会在 GitHub 创建 WebHook 来调用 Tekton 的 EventListener，使它能接收到仓库推送事件。

TriggerTemplate

当 EventListener 接收到外部事件之后，它会调用 Trigger 也就是触发器，而 TriggerTemplate 是用来定义接收到事件之后需要创建的 Tekton 资源的，例如创建一个 PipelineRun 对象来运行流水线。这节课，我们会使用 TriggerTemplate 来创建 PipelineRun 资源。

Step

Step 是流水线中的一个具体的操作，例如构建和推送镜像操作。Step 接收镜像和需要运行的 Shell 脚本作为参数，Tekton 将会启动镜像并执行 Shell 脚本。

Task

Task 是一组有序的 Step 集合，每一个 Task 都会在独立的 Pod 中运行，Task 中不同的 Step 将在同一个 Pod 不同的容器内运行。

Pipeline

Pipeline 是 Tekton 中的一个核心组件，它是一组 Task 的集合，Task 将组成一组有向无环图 (DAG)，Pipeline 会按照 DAG 的顺序来执行。

PipelineRun

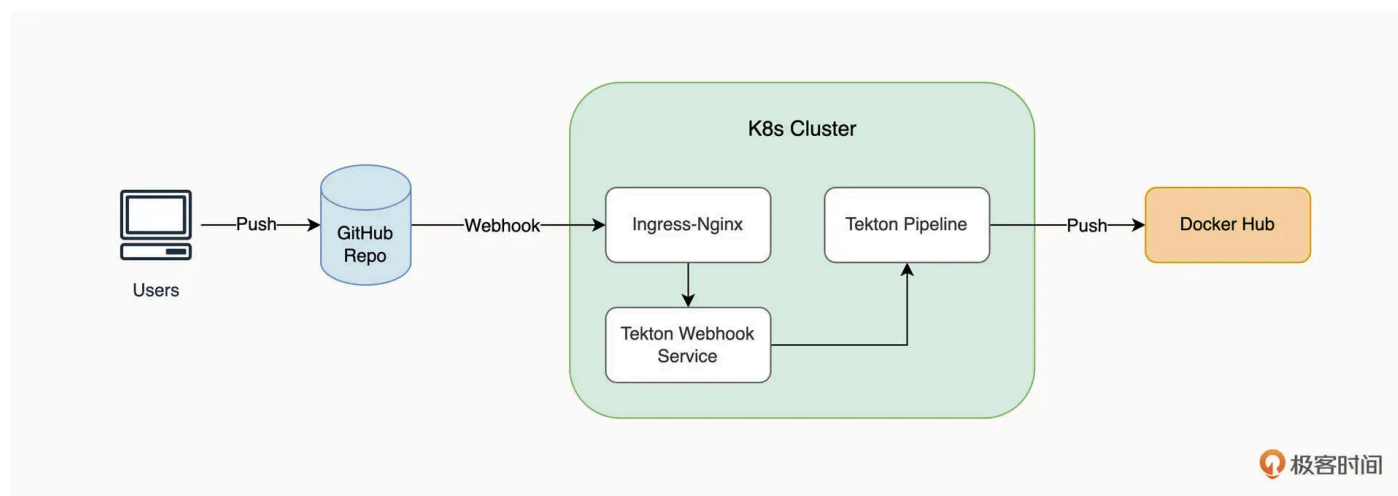
PipelineRun 实际上是 Pipeline 的实例化，它负责为 Pipeline 提供输入参数，并运行 Pipeline。例如，两次不同的镜像构建操作对应的就是两个不同的 PipelineRun 资源。

创建 Tekton Pipeline

可以看出，Tekton 的概念确实比较多，抽象也不好理解。别担心，接下来我们就实际创建一下流水线，在这个过程中不断加深理解。

概述

我们先来看看这节课创建的 Tekton 流水线最终可以实现的效果，如下图所示。



简单来说，当我们向 GitHub 推送代码时，GitHub 将以 HTTP 请求的方式通知集群内的 Tekton 触发器，触发器通过 Ingress-Nginx 对外暴露，当触发器接收到来自 GitHub 的事件推送时，将通过 TriggerTemplate 来创建 PipelineRun 运行 Pipeline，最终实现镜像的自动构建和推送。

创建 Task

好，下面我们正式开始实战。在创建 Pipeline 之前，我们需要先创建两个 Task，这两个 Task 分别负责“检出代码”还有“构建和推送镜像”。

首先创建检出代码的 Task。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 task.tekton.dev/git-clone created
```

这个 Task 是 Tekton 官方提供的插件，它和 GitHub Action 的 checkout 插件有一点类似，主要作用是检出代码。在这里，我们不需要理解它具体的细节。

然后，创建构建和推送镜像的 Task。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 task.tekton.dev/docker-socket configured
```

简单介绍一个这个 Task，关键内容如下。

 复制代码

```
1 apiVersion: tekton.dev/v1beta1
2 kind: Task
3 metadata:
4   name: docker-socket
5 spec:
6   workspaces:
7     - name: source
8   params:
9     - name: image
10       description: Reference of the image docker will produce.
11       .....
12   steps:
13     - name: docker-build
14       image: docker:stable
15     env:
```

```

16         .....
17         - name: IMAGE
18           value: $(params.image)
19         - name: DOCKER_PASSWORD
20           valueFrom:
21             secretKeyRef:
22               name: registry-auth
23               key: password
24         - name: DOCKER_USERNAME
25           valueFrom:
26             secretKeyRef:
27               name: registry-auth
28               key: username
29       workingDir: $(workspaces.source.path)
30       script: |
31         cd $SUBDIRECTORY
32         docker login $REGISTRY_URL -u $DOCKER_USERNAME -p $DOCKER_PASSWORD
33         if [ "${REGISTRY_URL}" = "docker.io" ] ; then
34             docker build --no-cache -f $CONTEXT/$DOCKERFILE_PATH -t $DOCKER_USERNAME
35             docker push $DOCKER_USERNAME/$IMAGE:$TAG
36             exit
37         fi
38         docker build --no-cache -f $CONTEXT/$DOCKERFILE_PATH -t $REGISTRY_URL/$RE
39         docker push $REGISTRY_URL/$REGISTRY_MIRROR/$IMAGE:$TAG
40       volumeMounts: # 共享 docker.socket
41         - mountPath: /var/run/
42           name: dind-socket
43       sidecars: #sidecar 提供 docker daemon
44         - image: docker:dind
45         .....

```

spec.params 字段用来定义变量，并最终由 PipelineRun 提供具体的值。

spec.steps 字段用来定义具体的执行步骤，例如，我们在这里使用 docker:stable 镜像创建了容器，并将 spec.params 定义的变量以 ENV 的方式传递到容器内部，其中 DOCKER_PASSWORD 和 DOCKER_USERNAME 两个变量来源于 Secret，我们将在后续创建。

spec.steps[0].script 字段定义了具体执行的命令，这里执行了 docker login 登录到 Docker Hub，并且使用了 docker build 和 docker push 来构建和推送镜像。我们对 Docker Hub 和其他镜像仓库做了区分，以便使用不同的 TAG 命名规则。

spec.sidecars 字段为容器提供 Docker daemon，它使用的镜像是 docker:dind。

仔细回想一下上节课的内容你会发现，**这个 Task 定义的具体行为和 GitLab CI 定义的流水线非常类似**，它们都是指定一个镜像，然后运行一段脚本，并且都是用 DiND 的方式来构建和推送镜像的。

创建 Pipeline

创建完 Task 之后，由于它们实现的具体功能是独立的，所以我们需要将他们联系起来。也就是说，我们希望 Pipeline 先克隆代码，再构建和推送镜像。所以，下面我们需要创建 Pipeline 来引用这两个 Task。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 pipeline.tekton.dev/github-trigger-pipeline created
```

这里我也简单介绍一下这个 Pipeline。

 复制代码

```
1 apiVersion: tekton.dev/v1beta1
2 kind: Pipeline
3 metadata:
4   name: github-trigger-pipeline
5 spec:
6   workspaces:
7     - name: pipeline-pvc
8     .....
9   params:
10     - name: subdirectory # 为每一个 Pipeline 配置一个 workspace，防止并发错误
11       type: string
12       default: ""
13     - name: git_url
14     .....
15   tasks:
16     - name: clone
17       taskRef:
18         name: git-clone
19       workspaces:
20         - name: output
```

```

21     workspace: pipeline-pvc
22     - name: ssh-directory
23     workspace: git-credentials
24     params:
25     - name: subdirectory
26       value: $(params.subdirectory)
27     - name: url
28       value: $(params.git_url)
29   - name: build-and-push-frontend
30     taskRef:
31       name: docker-socket
32     runAfter:
33     - clone
34     workspaces:
35     - name: source
36       workspace: pipeline-pvc
37     params:
38     - name: image
39       value: "frontend"
40     .....
41   - name: build-and-push-backend
42     taskRef:
43       name: docker-socket
44     runAfter:
45     - clone
46     workspaces:
47     - name: source
48       workspace: pipeline-pvc
49     params:
50     - name: image
51       value: "backend"
52     .....

```

首先，spec.workspaces 定义了一个工作空间。还记得我们提到的每一个 Task 都会在独立的 Pod 中运行吗？那么不同的 Task 如何共享上下文呢？答案就是 workspaces。实际上它是一个 PVC 持久化卷，这个 PVC 将会在 Pod 之间复用，这就让下游 Task 可以读取到上游 Task 写入的数据（比如克隆的代码）。

spec.params 定义了 Pipeline 的参数，参数的传递顺序是：PipelineRun->Pipeline->Task。

spec.tasks 定义了 Pipeline 引用的 Task，例如在这里分别引用了 git-clone 和 docker-socket 两个 Task，并且都指定了同一个 workspaces pipeline-pvc，然后指定了 params 向 Task 传递了参数值。

在 build-and-push-frontend 和 build-and-push-backend Task 中，都指定了 runAfter 字段，它的含义是等待 clone Task 执行完毕后再运行。

所以，Pipeline 对 Task 的引用就形成了一个有向无环图 (DAG)，在这个 Pipeline 中，首先会检出源码，然后以并行的方式同时构建前后端镜像。

创建 EventListener

创建完 Pipeline 之后，工作流实际上就已经定义好了。但是我们并不希望手动来运行它，我们希望通过 GitHub 来自动触发它。所以，接下来需要创建 EventListener 来获得一个能够监听外部事件的服务。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 eventlistener.triggers.tekton.dev/github-listener created
```

EventListener 的具体作用是：接收来自 GitHub 的 Webhook 调用，并将 Webhook 的参数和 TriggerTemplate 定义的参数对应起来，以便将参数值从 Webhook 一直传递到 PipelineRun。

暴露 EventListener

在 EventListener 创建完成之后，Tekton 将会拉起一个 Deployment 用来处理 Webhook 请求，你可以通过 kubectl get deployment 命令来查看。

[复制代码](#)

```
1 $ kubectl get deployment
2 NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
3 el-github-listener                 1/1      1              1            22m
```

同时，Tekton 也会为 el-github-listener Deployment 创建 Service，以便接受来自外部的 HTTP 请求。

[复制代码](#)

```
1 $ kubectl get service
2 NAME                                TYPE                CLUSTER-IP          EXTERNAL-IP        PORT(S)
3 el-github-listener                 ClusterIP            172.16.253.54       <none>              8080/TCP,9000
```

为了能够让 GitHub 将事件推送到 Tekton 中，我们需要暴露 el-github-listener Service。我使用了 Ingress-Nginx 来对外暴露它。

[复制代码](#)

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 ingress.networking.k8s.io/ingress-resource created
```

这个 Ingress 的内容比较简单，具体如下。

[复制代码](#)

```
1 apiVersion: networking.k8s.io/v1
2 kind: Ingress
3 metadata:
4   name: ingress-resource
5   annotations:
6     kubernetes.io/ingress.class: nginx
7     nginx.ingress.kubernetes.io/ssl-redirect: "false"
8 spec:
9   rules:
10    - http:
11      paths:
12        - path: /hooks
```

```
13         pathType: Exact
14         backend:
15             service:
16                 name: el-github-listener
17                 port:
18                     number: 8080
```

在 Ingress 策略中，我们没有使用 host 定义域名，而使用了 path 来匹配路径。这样，Tekton 接收外部 Webhook 的入口也就是 Ingress-Nginx 的负载均衡器 IP 地址了，具体的地址为 <http://43.135.82.249/hooks>。

创建 TriggerTemplate

不过 EventListener 并不能独立工作，它还需要一个助手，那就是 TriggerTemplate。TriggerTemplate 是真正控制 Pipeline 启动的组件，它负责创建 PipelineRun。

我们可以通过下面的命令来创建 TriggerTemplate。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 triggertemplate.triggers.tekton.dev/github-template created
```

创建 Service Account 和 PVC

下一步，由于触发器并没有具体的执行用户，所以我们还需要为触发器配置权限，也就是创建 Service Account。同时，我们也可以一并创建用于共享 Task 之间上下文的 PVC。

 复制代码

```
1 $ kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1
2 serviceaccount/tekton-build-sa created
3 clusterrolebinding.rbac.authorization.k8s.io/tekton-clusterrole-binding created
4 persistentvolumeclaim/pipeline-pvc created
5 role.rbac.authorization.k8s.io/tekton-triggers-github-minimal created
6 rolebinding.rbac.authorization.k8s.io/tekton-triggers-github-binding created
7 clusterrole.rbac.authorization.k8s.io/tekton-triggers-github-clusterrole created
8 clusterrolebinding.rbac.authorization.k8s.io/tekton-triggers-github-clusterbindin
```

设置 Secret

最后，我们还需要为 Tekton 提供一些凭据信息，例如 Docker Hub Token、GitHub Webhook Secret 以及用于检出私有仓库的私钥信息。

将下面的内容保存为 `secret.yaml`，并修改相应的内容。

 复制代码

```
1 apiVersion: v1
2 kind: Secret
3 metadata:
4   name: registry-auth
5   annotations:
6     tekton.dev/docker-0: https://docker.io
7 type: kubernetes.io/basic-auth
8 stringData:
9   username: "" # docker username
10  password: "" # docker hub token
11
12 ---
13 # github webhook token secret
14 apiVersion: v1
15 kind: Secret
16 metadata:
17   name: github-secret
18 type: Opaque
19 stringData:
20   secretToken: "webhooksecret"
21 ---
22 apiVersion: v1
23 kind: Secret
24 metadata:
```

```
25   name: git-credentials
26   data:
27     id_rsa: LS0tLS.....
28     known_hosts: Z2l0aHVlM.....
29     config: SG9zd.....
```

解释一下，你主要需要修改的是下面这几个字段。

将 `stringData.username` 替换为你的 Docker Hub 的用户名。

将 `stringData.password` 替换为你的 Docker Hub Token，如果你还没有创建好，可以在 [🔗 上一节课](#) 找到相应内容。

将 `data.id_rsa` 替换为你本地 `~/.ssh/id_rsa` 文件的 base64 编码内容，这将会为 Tekton 提供检出私有仓库的权限，你可以使用 `$ cat ~/.ssh/id_rsa | base64` 命令来获取。

将 `data.known_hosts` 替换为你本地 `~/.ssh/known_hosts` 文件的 base64 编码内容，你可以通过 `$ cat ~/.ssh/known_hosts | grep "github" | base64` 命令来获取。

将 `data.config` 替换为你本地 `~/.ssh/config` 文件的 base64 编码内容，你可以通过 `$ cat ~/.ssh/config | base64` 命令来获取。

然后，运行 `kubectl apply`，同时将这 3 个 Secret 应用到集群内。

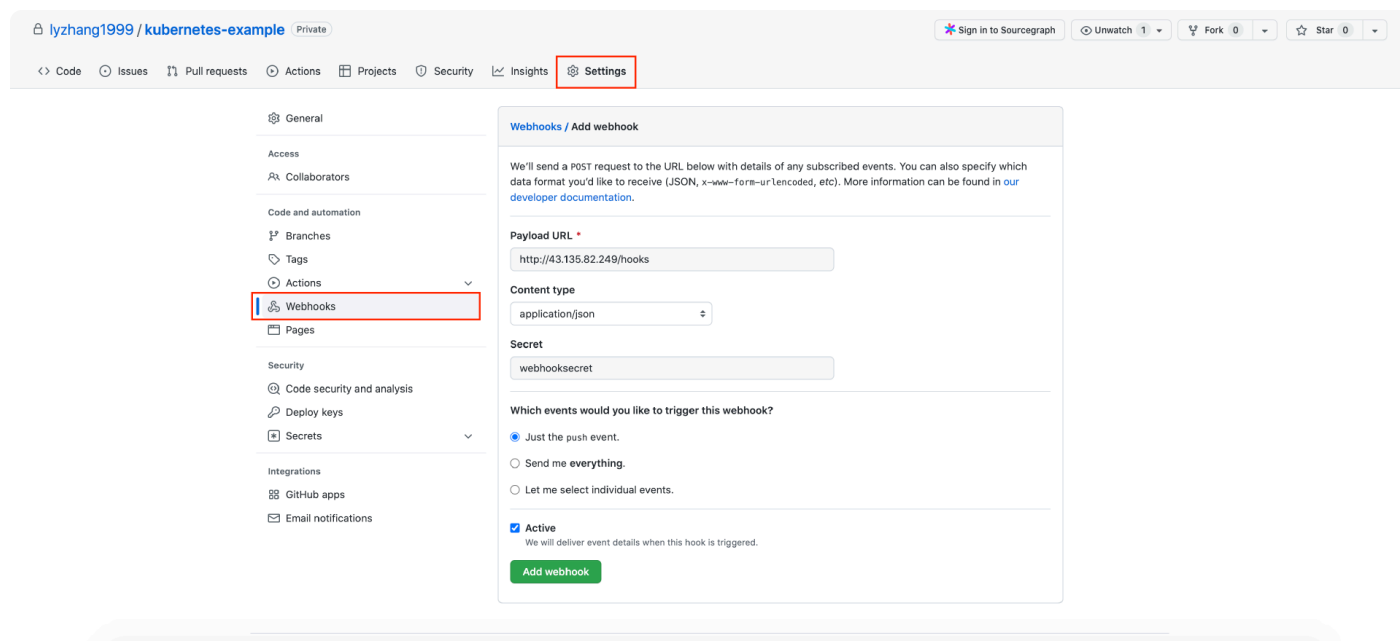
 复制代码

```
1 $ kubectl apply -f secret.yaml
2 secret/registry-auth created
3 secret/github-secret created
4 secret/git-credentials created
```

创建 GitHub Webhook

到这里，Tekton 的配置就已经完成了。接下来还剩最后一步：创建 GitHub Webhook。

打开你在 GitHub 创建的 kubernetes-example 仓库，进入“Settings”页面，点击左侧的“Webhooks”菜单，在右侧的页面中按照下图进行配置。



输入 Webhook 地址，也就是“Ingress-Nginx 网关的负载均衡器地址 + hooks 路径”，并且将“Content type”配置为“application/json”。

点击“Add webhook”创建。

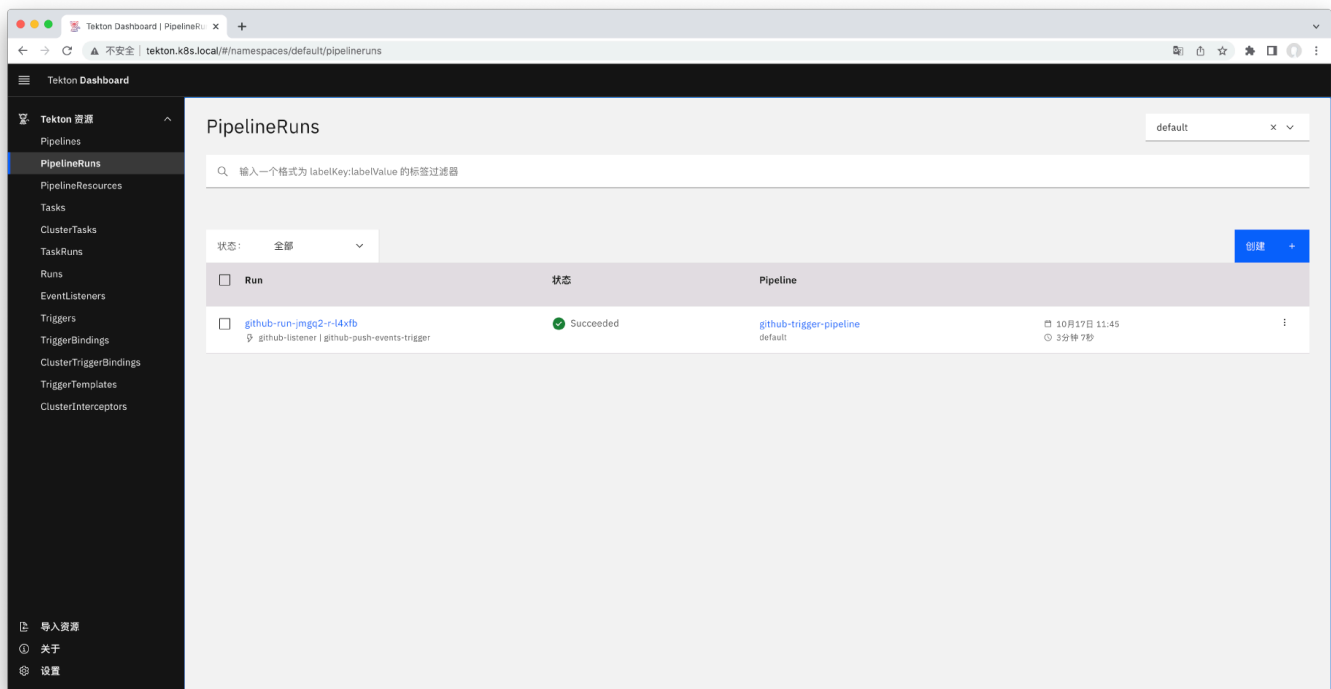
触发 Pipeline

到这里，所有的准备工作就已经完成了。现在我们向仓库提交一个空的 commit 来触发 Pipeline。

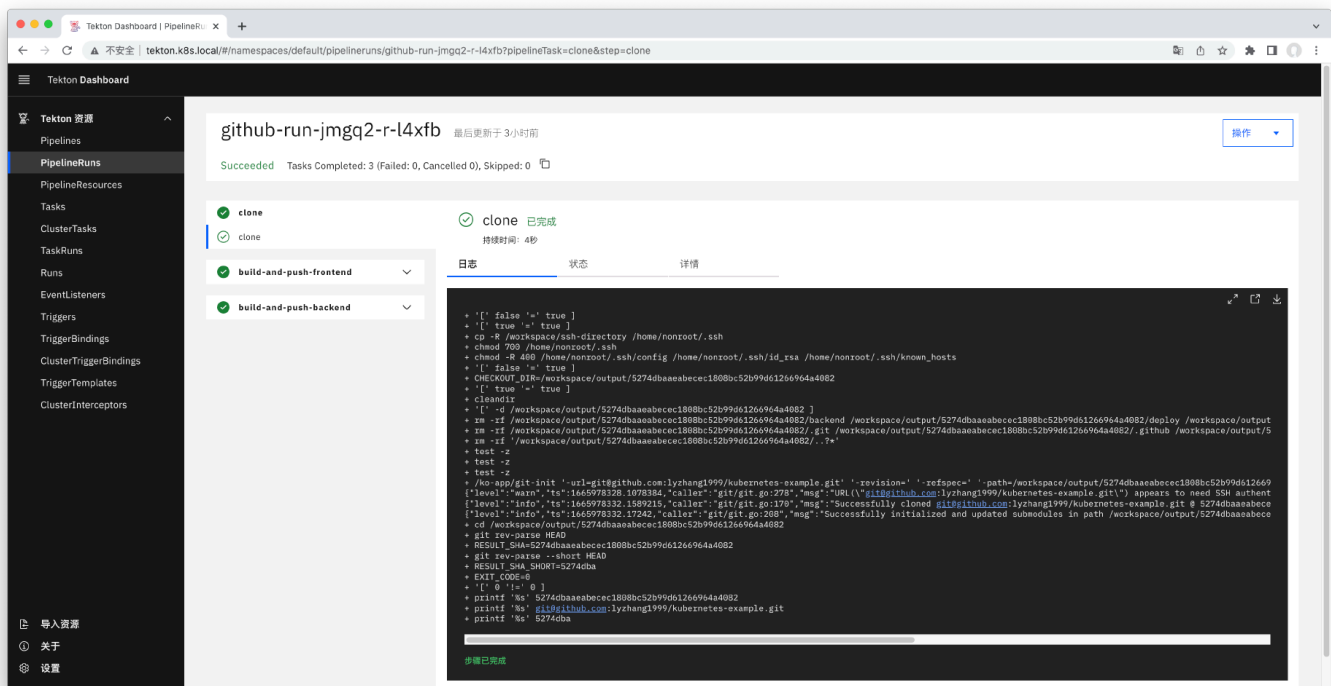
 复制代码

```
1 $ git commit --allow-empty -m "Trigger Build"
2 [main 79ca67e] Trigger Build
3 $ git push origin main
```

完成推送后，打开 <http://tekton.k8s.local/> 进入 Tekton 控制台，点击左侧的“PipelineRun”，你会看到刚才触发的 Pipeline。



点击 PipelineRun 的名称进入详情页面，可以看到 Pipeline 包含 3 个 Task 及其输出的日志信息。



到这里，我们就成功地将 GitHub 和 Tekton Pipeline 结合，并通过 GitHub Webhook 触发了 Pipeline，实现了镜像的自动构建和推送，和上一节课实现的效果一样，每次有新的提交时

都会触发构建，并且每个 commit id 对应一个镜像版本。

总结

总结一下。在这节课，我们介绍了如何通过 Tekton Pipeline 来自动构建和推送镜像。

Tekton 的概念繁多，并且比较抽象，完全掌握这些概念有一定难度。为了方便理解和记忆，你可以把 Tekton 的概念和 GitHub Action 做类比，例如，Task 有点像 GitHub Action 的插件，git-clone Task 实际上和我们在 GitHub Action 引用的 actions/checkout@v3 插件功能是类似的。

其次，Tekton Pipeline 和 GitHub Action Workflow 也比较类似，Tekton Pipeline 通过引用和组合不同的 Task 形成了一个流水线。而 GitHub Action Workflow 则是通过引用并组合插件来完成一个工作流。

相比较而言，它们之间最大的差异还是在触发器上。GitHub Action Workflow 的触发器内置到了工作流中，只需要在工作流声明什么时候可以被触发即可。而 Tekton 则需要借助 EventListener 和 TriggerTemplate 这两个组件来接收外部 Webhook 事件并触发 Pipeline。所以 Tekton 复杂的配置也主要集中在这两块。

Tekton 虽然配置相对复杂，但它是一次性配置，如果你已有 Kubernetes 集群，那么使用 Tekton 来构建镜像是一个不错的选择，这种方案使得镜像构建成本几乎为零，并且扩展性强。

在前面几节课，我们都是使用 Docker Hub 作为镜像存储仓库的。但实际上，它同样也需要收费，在镜像存储方面，我们同样也可以使用自托管方案来进一步降低成本。

在下一节课，我们会详细介绍这部分的内容。

课后题

最后，给你留一道思考题吧。

在一个 Tekton Task 中，Step 是在同一个 Pod 不同的容器里运行的，我们知道 Pod 的容器启动是没有顺序的，那么 Tekton 是如何控制 Step 的运行顺序的呢？

提示：你可以在 Pipeline 运行时，查看 Task 拉起的 Pod Manifest 查找线索（`kubectl get pods $POD_NAME -o yaml`）。

欢迎你给我留言交流讨论，你也可以把这节课分享给更多的朋友一起阅读。我们下节课见。

AI智能总结

Tekton 是一款基于 Kubernetes 的开源 CI/CD 产品，适用于构建次数频繁的中大型团队。本文详细介绍了使用 Tekton 构建镜像的自托管方案。首先介绍了准备 Kubernetes 集群的步骤，包括在云厂商创建集群、安装 Tekton 相关组件和 Ingress-nginx。随后深入讲解了 Tekton 的基本概念，包括 EventListener、TriggerTemplate、PipelineRun、Pipeline、Task 和 Step，并通过示例演示了如何配置 Tekton 的流水线，并结合 GitHub 配置 Webhook 触发器，实现提交代码后触发 Tekton 流水线并构建镜像，最后推送到镜像仓库的过程。读者通过学习本文可以掌握 Tekton 的流水线和触发器的用法，具备独立配置的能力。整体内容详实，适合需要了解 Tekton 构建镜像自托管方案的技术人员阅读。Tekton 的概念繁多，并且比较抽象，完全掌握这些概念有一定难度。为了方便理解和记忆，你可以把 Tekton 的概念和 GitHub Action 做类比，例如，Task 有点像 GitHub Action 的插件，git-clone Task 实际上和我们在 GitHub Action 引用的 actions/checkout@v3 插件功能是类似的。其次，Tekton Pipeline 和 GitHub Action Workflow 也比较类似，Tekton Pipeline 通过引用和组合不同的 Task 形成了一个流水线。而 GitHub Action Workflow 则是通过引用并组合插件来完成一个 workflow。Tekton 虽然配置相对复杂，但它是一次性配置，如果你已有 Kubernetes 集群，那么使用 Tekton 来构建镜像是一个不错的选择，这种方案使得镜像构建成本几乎为零，并且扩展性强。在前面几节课，我们都是使用 Docker Hub 作为镜像存储仓库的。但实际上，它同样也需要收费，在镜像存储方面，我们同样也可以使用自托管方案来进一步降低成本。

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (15)

最新 精选



李多

2023-01-26 来自山东

原文：

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/dashboard/latest/tekton-dashboard-release.yaml
```

貌似已经 404 无法访问了。

新的 dashboard yaml 在这个路径下：

```
kubectl apply --filename https://storage.googleapis.com/tekton-releases/dashboard/latest/release.yaml
```

参照：<https://github.com/tektoncd/dashboard>

作者回复：感谢分享，原文已修改。

共 3 条评论 >

👍 5



Waylon

2023-01-20 来自北京

容器的entrypoint启动进程

监控到/tekton/downward/ready文件的创建, 并等待文件内容的写入

执行git-init子进程, 从 git 仓库克隆源码

创建/tekton/tools/0文件

所有的 Step 都是被 /tekton/tools/entrypoints 封装起来执行的。

-wait_file 指定一个文件，通过监听文件句柄，在探测到文件存在时执行被封装的 Step 任务。

-post_file 指定一个文件，在Step任务完成后创建这个文件。

通过文件序列 /tekton/tools/\${index} 来对 Step 进行排序。

作者回复：👍 课代表！

共 2 条评论 >

👍 3



ghostwritten

2023-01-30 来自北京

跟着步骤已全部实现，学到了不少东西。

第一页：

1. 腾讯云创建k8级群：申请标准集群、创建集群网络、设置配额、外网访问、本地kubectl访问集群。

2. 安装 Tekton Operator

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/pipeline/latest/release.yaml
```

```
等待：kubectl wait --for=condition=Ready pods --all -n tekton-pipelines --timeout=300s
```

3. 安装 Tekton Trigger 和 Tekton Interceptors 组件

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest/release.yaml
```

```
kubectl apply -f https://storage.googleapis.com/tekton-releases/triggers/latest/interceptors.yaml
```

```
kubectl wait --for=condition=Ready pods --all -n tekton-pipelines --timeout=300s
```

4. 安装 Ingress-Nginx

```
kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.4.0/deploy/static/provider/cloud/deploy.yaml
```

```
kubectl wait --for=condition=AVAILABLE deployment/ingress-nginx-controller --all -n ingress-nginx
```

5. 暴露 Tekton Dashboard

```
kubectl apply -f tekton-dashboard.yaml
```

获取外网 IP : `kubectl get services --namespace ingress-nginx ingress-nginx-controller --output jsonpath='{.status.loadBalancer.ingress[0].ip}'`

配置43.135.82.249 tekton.k8s.local 通过域名 `http://tekton.k8s.local` 来访问Tekton Dashboard

6. 创建 Task: 检出代码

```
kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1999/gitops/main/ci/18/tekton/task/git-clone.yaml
```

7. 创建 Task: 构建和推送镜像

```
kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1999/gitops/main/ci/18/tekton/task/docker-build.yaml
```

8. 创建 Pipeline

```
kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1999/gitops/main/ci/18/tekton/pipeline/pipeline.yaml
```

9. 创建 EventListener

```
kubectl apply -f https://ghproxy.com/https://raw.githubusercontent.com/lyzhang1999/gitops/main/ci/18/tekton/trigger/github-event-listener.yaml
```

作者回复: 赞, 很高兴看到你获得了收获! 加油~



1



邵涵

2023-03-01 来自北京

如果按照本文的方式对多个github仓库使用Tekton配置自动构建镜像能力, 是需要针对每个github仓库都创建独立的webhook url (并在ingress中为其做相应配置) 并创建独立的EventListener、TriggerTemplate、Pipeline等对象吗? 因为看起来这些对象之间是环环相扣的并在yaml定义中有针对比如镜像名称的一些信息的hardcode的, 这样看起来就无法一套对象对多个git仓库通用了

作者回复: 是的, 只有 Task 可以复用。如果有很多仓库需要构建镜像的话, 也可以考虑用 GitHub Action。不过不同的仓库实际上也需要重新写 Action YAML, 只不过它可以内置很多变量可以直接使用, 这样让 YAML 的内容看起来都差不多。



老虎

2023-02-26 来自北京

请问这个secret里面`cat ~/.ssh/known_hosts | grep "github" | base64`找不到github的known_host值, 应该怎么处理, 谢谢

作者回复: 先在 github 配置一下 ssh key: <https://docs.github.com/en/authentication/connecting-to-github-with-ssh/adding-a-new-ssh-key-to-your-github-account>



Geek_5dadb9

2023-02-21 来自北京

`cat ~/.ssh/known_hosts | grep "github" | base64` 和 `cat ~/.ssh/config | base64` 没有找到 是哪里没有配置吗?

作者回复: 你的本机需要先从 GitHub 克隆一个仓库。

共 2 条评论 >





Geek_03197d

2023-02-12 来自江苏

Tekton & skaffold 能否对比一下？哪个业界用的更广泛？（我看skaffold的github 上的star更多），谢谢

作者回复: Skaffold 在本地开发的时候比较常用，Tekton 通用型会更好一些。



gyl1989113

2023-02-08 来自四川

Failed to pull image "gcr.io/tekton-releases/github.com/tektoncd/pipeline/cmd/controller:v0.44.0@sha256:51e4c16fc4d0b18912106ff7fe37cc24c0667de6af468cdd1a9e9dc174039de1": rpc error: code = Unknown desc = failed to pull and unpack image "gcr.io/tekton-releases/github.com/tektoncd/pipeline/cmd/controller@sha256:51e4c16fc4d0b18912106ff7fe37cc24c0667de6af468cdd1a9e9dc174039de1": failed to resolve reference "gcr.io/tekton-releases/github.com/tektoncd/pipeline/cmd/controller@sha256:51e4c16fc4d0b18912106ff7fe37cc24c0667de6af468cdd1a9e9dc174039de1": failed to do request: Head "https://gcr.io/v2/tekton-releases/github.com/tektoncd/pipeline/cmd/controller/manifests/sha256:51e4c16fc4d0b18912106ff7fe37cc24c0667de6af468cdd1a9e9dc174039de1": dial tcp 108.177.125.82:443: i/o timeout. 咋办呢

作者回复: gcr.io 的网络问题，建议开通一台云厂商的香港集群来测试，或者解决无法访问的问题。



Amos

2023-01-21 来自江苏

创建 Task和创建 Pipeline两步中各有两个是spec，而不是spce吧？

作者回复: 感谢指正~



Amos

2023-01-21 来自江苏

tekton是否支持图形化拖拽的方式编辑task、piplinerun呢？

作者回复: 不支持哦，只能通过 YAML 来配置。

